Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

"Linux Process Tree Visualizer: A Command-Line Tool Using the /proc Filesystem and Process Management System Calls"

Course: Operating Systems and Systems Programming (25CS2104E), 2026–27, Term-I

Section 13 | Team 17

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

S UDAYASRI	**2520090003**
B AKSHITHA	**2520090014**
R BHANU PRIYA	**2520090171**

Under the Esteemed Guidance of

S Alekhya

Faculty, Department of Computer Science and Engineering

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

2026–2027

Declaration

The Project Report entitled "Linux Process Tree Visualizer: A Command-Line Tool Using the /proc Filesystem and Process Management System Calls" is a record of Bonafide work of S UDAYASRI - 2520090003, B AKSHITHA - 2520090014, R BHANU PRIYA - 2520090171 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

S UDAYASRI	**2520090003**
B AKSHITHA	**2520090014**
R BHANU PRIYA	**2520090171**

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled "Linux Process Tree Visualizer: A Command-Line Tool Using the /proc Filesystem and Process Management System Calls" is a bonafide work done and submitted by S Udayasri – 2520090003, B Akshitha – 2520090014, R Bhanu Priya – 2520090171 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science and Engineering, K L (Deemed to be University), during the academic year 2026–2027.

Signature of the Guide **Signature of the Course Coordinator** **Signature of the HOD**

S Alekhya

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our guide S Alekhya, Department of Computer Science and Engineering, whose tremendous support, encouragement and guidance throughout the Operating Systems and Systems Programming project cannot be expressed in words. We are also grateful to our Course Coordinator [Course Coordinator Name] for the direction given during the course.

We express our gratitude to [HOD Name], Head of the Department of Computer Science and Engineering, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

S UDAYASRI	**2520090003**
B AKSHITHA	**2520090014**
R BHANU PRIYA	**2520090171**

Abstract

This project aims to design and implement a Linux Process Tree Visualizer using core Operating Systems and Systems Programming concepts. In Linux every process is created by another process, so all running programs form a tree that starts at the first process (PID 1) and grows through the fork() system call. Commands such as ps show this information only as a flat table, which makes it hard for a learner to see who created whom. This project builds a lightweight command-line tool that reads the process information kept by the kernel and draws the parent–child hierarchy as a tree in the terminal.

The tool is written in C for Linux/Ubuntu using POSIX APIs. It scans the /proc filesystem with opendir() and readdir(), reads the file /proc/PID/stat of every process with open() and read(), and extracts the PID, parent PID (PPID), name, state, number of threads and resident memory. The processes are sorted by PID and linked into a tree using a first-child / next-sibling structure, and the tree is printed with a recursive depth-first traversal. Options allow the user to show extra details (-d), display only the subtree of one process (-r), use plain ASCII lines (-a) and refresh the tree continuously (-w). A built-in demo mode (-D) uses fork() and waitpid() to create a small process tree that includes a zombie process, and a SIGINT handler installed with sigaction() stops the live mode cleanly.

The outcome is a working command-line process tree visualizer that shows the complete hierarchy of running processes, highlights zombie processes, prints a summary of the number of processes and the depth of the tree, and gives hands-on understanding of key Operating Systems concepts such as the process control block, parent–child relationships, process states, the /proc filesystem, system calls and signals.

Keywords: Linux, Process Tree, /proc Filesystem, PID, PPID, fork(), Zombie Process, System Calls, Signals.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	**Introduction**		1–2
		1.1 Background of the Project	1
		1.2 Problem Statement	1
		1.3 Scope of the Project	1
		1.4 Objectives of the Project	2
		1.5 Importance of the Application	2
		1.6 Target Users / Audience	2
2	**OS Concepts and Background Study**		3–4
		2.1 Operating System Concepts Related to the Project	3
		2.2 Existing Methods / Algorithms	3
		2.3 Proposed Approach	4
		2.4 Advantages of the Proposed Approach	4
3	**System Requirements**		5–6
		3.1 Hardware Requirements	5
		3.2 Software Requirements	5
		3.3 Operating System Environment – Ubuntu / WSL	5
		3.4 Development Tools and Compilers	6
4	**Technology and Tools Used**		7–8
		4.1 C / C++ Programming	7
		4.2 GCC Compiler	7
		4.3 Linux System Calls	7
		4.4 POSIX Threads / Synchronization	7
		4.5 Linux Commands and Utilities	8
		4.6 Other Tools / Libraries Used	8
5	**System Architecture and Design**		9–11
		5.1 Overall System Architecture	9
		5.2 System Workflow	9
		5.3 Flowchart / Data Flow	10
		5.4 Module Description	10
		5.5 Input and Output Design	11
6	**Implementation**		12–19
		6.1 Module-wise Implementation	12
		6.2 Algorithm / Pseudocode	13

S. No.	Chapter	Topics / Subtopics	Page No.
		6.3 Core C / C++ Implementation	13
		6.4 Linux System Calls / APIs Used	17
		6.5 Process / Thread Implementation	17
		6.6 IPC / Synchronization Implementation	18
		6.7 Memory Management Implementation	18
		6.8 Optional User Interface Implementation	19
7	**Execution and Demonstration**		20–23
		7.1 Compilation in Ubuntu	20
		7.2 Execution Commands	20
		7.3 Sample Inputs and Outputs	20
		7.4 Screenshots of Execution	22
		7.5 Demonstration of OS Concepts	22
8	**Conclusion**		24–25
		8.1 Summary of the Project	24
		8.2 Objectives Achieved	24
		8.3 Results Achieved	24
		8.4 Skills and Learning Outcomes	24
9	**References**		26
		9.1 Books	26
		9.2 Research Papers	26
		9.3 Linux / GNU Documentation	26
		9.4 Online Resources	26

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Every program that runs on a Linux computer is a process, and every process (except the very first one) is created by another process. When you open a terminal, the terminal is a process; the shell inside it is its child; and every command you type becomes a child of the shell. All these parent–child links together form a tree, the process tree, whose root is the first process started by the kernel (PID 1, usually systemd or init).

Most users never see this tree. Tools such as ps and top present processes as a long flat list, so the idea of "who created whom", which is central to the process-management part of the Operating Systems course, stays hidden. At the same time the kernel exposes all this information through a virtual filesystem called /proc, where each running process has its own folder.

This project, "Linux Process Tree Visualizer", is a lightweight command-line tool for Linux (Ubuntu) written in C using the POSIX APIs. Instead of calling ps or any ready-made library, it reads /proc itself with system calls such as opendir(), readdir(), open() and read(), builds the tree in memory and prints it in the terminal. It also contains a demo mode that creates a real process tree with fork() so that the learner can see the tree appear, including a zombie process. In this way the project turns an everyday idea into a practical demonstration of process management, the /proc filesystem, system calls and signals.

The project was carried out as part of the course Operating Systems and Systems Programming (25CS2104E), Term-I of 2026–27, by Team 17 of Section 13.

1.2 Problem Statement

Process-listing commands such as ps show processes as a flat table, and graphical system monitors hide the underlying mechanisms. As a result it is difficult for a learner to understand how processes are related to each other, how a process is created by fork(), what the process states mean, and how the operating system keeps this information.

There is a need for a simple, Linux-based tool that directly uses OS-level interfaces (the /proc filesystem and system calls) to collect process information and to display the parent–child hierarchy of all running processes as a tree, demonstrating practical applications of process management and system programming concepts.

1.3 Scope of the Project

The scope of the project is limited to a command-line process tree visualizer for Linux/Ubuntu, implemented in C. Within this scope the application covers:

- Reading every running process from the /proc filesystem (PID, parent PID, name, state, number of threads and resident memory).
- Building the parent–child tree of all processes in memory, sorted by PID.
- Printing the tree in the terminal with box-drawing lines (or plain ASCII lines with the -a option) together with a summary line.

- Showing the subtree of one chosen process (-r PID) and optional details for each process (-d).
- Highlighting zombie (defunct) processes in the tree.
- A live mode (-w seconds) that refreshes the tree periodically and stops cleanly on Ctrl+C using a SIGINT handler.
- A demo mode (-D) that uses fork() and waitpid() to create a small process tree containing a zombie process.

The following are outside the scope of this version: a graphical interface, sending signals to or killing processes from the tool, CPU-percentage and I/O statistics, per-thread listing, container/namespace views and monitoring of other computers. These are listed as possible future enhancements.

1.4 Objectives of the Project

- To design a Linux-based command-line process tree visualizer using C and POSIX system calls.
- To collect process information directly from the /proc filesystem using directory and file system calls.
- To build and display the parent–child hierarchy of processes using a tree data structure and recursive traversal.
- To demonstrate process creation and termination (fork, wait, zombie state) and signal handling (SIGINT) through the demo and live modes.

1.5 Importance of the Application

- Educational value: it connects theory taught in the OS course (process control block, PID/PPID, process states, fork/wait, zombies, /proc) with a program whose output can be seen and verified immediately.
- Clarity: a tree shows ownership and origin of every process at a glance, which a flat list cannot do; for example it is easy to see which shell started which command.
- Efficiency: the tool takes a single snapshot of /proc and uses a few kilobytes of memory, with no external libraries.
- Troubleshooting aid: the same view is used by system administrators to find where a runaway process came from or which parent has left zombie children behind.

1.6 Target Users / Audience

- Students of Operating Systems and Systems Programming who want a hands-on example of processes, /proc, system calls and signals.
- Linux users and administrators who want a light terminal tool to see how processes are related.
- Faculty and evaluators looking for a compact project that clearly demonstrates OS concepts.
- Beginners in C systems programming who want to study a small, modular and well-commented code base.

CHAPTER 2: OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

The project is built on the following Operating System concepts. Each one is used for a specific purpose in the application.

OS Concept / API	Description	Use in the Project
Process and PCB (task_struct)	A process is a program in execution. The kernel keeps a Process Control Block for each process (task_struct in Linux) holding its PID, parent, state, memory and other data.	The tool shows the PCB fields that are visible to users: PID, PPID, name, state, threads and memory.
PID and PPID	Every process has a unique process ID (PID) and stores the ID of the process that created it (PPID).	The PPID of each process is the link used to join child nodes to parent nodes.
Process tree and init	Processes form a tree. PID 1 (systemd/init) is started by the kernel and is the ancestor of user processes; kthreadd (PID 2) is the parent of kernel threads.	These processes have no parent in the list and become the roots of the printed tree.
fork(), wait(), exit()	fork() creates a child copy of the calling process; exit() ends a process; wait()/waitpid() lets the parent collect the exit status of a child.	Used in the demo mode to build a real parent–child–grandchild tree.
Process states	A process is Running (R), Sleeping (S), in uninterruptible sleep (D), Stopped (T), Idle (I) or Zombie (Z).	The state letter is read from /proc/PID/stat and shown with the -d option.
Zombie and orphan processes	A zombie is a finished child whose parent has not yet called wait(); its PCB entry remains. An orphan is a child whose parent has ended; it is adopted by init (or a sub-reaper).	Zombies are marked “ZOMBIE” in the output; orphans appear under PID 1.
/proc filesystem	A virtual filesystem created by the kernel in memory. Each running process has a folder /proc/PID with files such as stat and status.	The main data source of the tool; no data is stored on disk.
Threads	A process can contain several threads that share its address space. Linux reports the number of threads in the stat file.	The thread count of each process is shown with -d.
Signals (SIGINT)	A signal is a software interrupt delivered asynchronously by the kernel to a process.	Ctrl+C sends SIGINT; a handler stops the live mode cleanly.
Virtual memory (RSS)	Resident Set Size is the part of a process’s memory that is currently in RAM, counted in pages.	RSS pages are converted to KB using sysconf(_SC_PAGESIZE).
File and directory system calls	opendir(), readdir(), open(), read() and close() access directories and files through the kernel.	Used to list /proc and to read the stat file of each process.

Information flows in this project from the kernel to the program: the kernel updates its process table, presents it as files in /proc, and the program reads those files and converts the text into its own tree of structures. The only other communication is a signal (SIGINT) from the user to the program.

2.2 Existing Methods / Algorithms

Several tools and approaches are already used to look at processes:

- ps: prints a flat table of processes; the PPID column shows the parent, but the user has to follow the links manually. With options such as ps axjf it can draw a simple forest.
- pstree: a ready-made tool that prints the process tree. It is very good, but it hides how the tree is built, so it does not show a learner the system calls and data structures involved.
- top / htop: interactive monitors that focus on CPU and memory usage; htop has a tree view but is a large program that uses a screen-drawing library.
- Reading /proc by hand (cat /proc/PID/stat): shows the raw data of a single process, which is difficult to interpret for a whole system.
- Graphical system monitors: convenient, but they depend on large libraries and abstract away the OS interfaces the course aims to teach.

2.3 Proposed Approach

The proposed solution reads the kernel's process information directly and converts it into a tree in four simple steps:

- Scan: open the directory /proc and, for every folder whose name is a number, read /proc/PID/stat with open() and read(). The process name is taken from the text between the first "(" and the last ")", because names may contain spaces.
- Store: keep the data of every process in a dynamically growing array of structures, which is doubled with realloc() when full.
- Link: sort the array by PID with qsort(); for each process find its parent with bsearch() using the PPID and attach it using first-child / next-sibling indices. Processes whose parent is not in the list (PID 1 and kthreadd) are the roots.
- Print: traverse the tree recursively (depth-first) and print each process with a prefix made of vertical and branch lines, then print a summary line (number of processes, zombies and depth of the tree).

For learning purposes two extra modes are added: a demo mode that creates a tree with fork(), and a live mode that refreshes the tree with nanosleep() until the user presses Ctrl+C (SIGINT).

2.4 Advantages of the Proposed Approach

- Transparency: every step uses a visible OS mechanism (directory reading, file reading, fork, signals) rather than a hidden library.
- Low resource usage: a single scan of /proc and about 96 bytes of memory per process; there are no external libraries or background threads.
- Complete view: the tree contains all processes including kernel threads, with the option to show only one subtree.
- Zombie detection: finished-but-not-reaped children are marked directly in the tree.
- Robustness: processes that finish while the scan is running are simply skipped instead of causing an error; names are copied into a fixed buffer with a length limit.

- Modular and maintainable code: separate functions for scanning, tree building, printing and demo make the program easy to extend.

CHAPTER 3: SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Minimum	Recommended
Processor	Dual-core, 1.0 GHz	Quad-core, 2.0 GHz or higher
RAM	1 GB	4 GB or more
Hard disk space	100 MB free (for tools and source)	1 GB free
Display / Input	Monitor, keyboard	Monitor, keyboard (terminal with UTF-8 support for the line-drawing characters)

The program itself is very small (about 200 lines of source and a binary of roughly 22 KB); the hardware requirements are mainly those of the Linux environment and development tools.

3.2 Software Requirements

Software	Details
Operating System	Ubuntu 20.04 LTS or later (native installation, virtual machine or WSL2)
Language	C (C11 with POSIX.1-2008 features)
Compiler	GCC 9 or later
Build tool	GNU Make
Libraries	glibc only (no extra libraries are needed)
Editor / IDE	VS Code or gedit
Version control	Git and GitHub

3.3 Operating System Environment – Ubuntu / WSL

The application is developed and tested on Ubuntu Linux, a Debian-based distribution that follows the POSIX standard and provides the /proc filesystem. It can be run in any of the following ways:

- Native Ubuntu installation on a laptop or desktop.
- Ubuntu in a virtual machine (VirtualBox or VMware).
- Windows Subsystem for Linux (WSL2) with an Ubuntu distribution, which provides a real Linux kernel on Windows. (The tree then shows only the Linux processes of that distribution, not Windows processes.)

The required packages are installed with the following commands:

```
sudo apt update
sudo apt install build-essential gdb git strace psmisc
```

The build-essential package provides GCC, make and the C library headers needed for the directory, process and signal functions. The psmisc package provides the pstree command, which is used only to cross-check the output of the project.

3.4 Development Tools and Compilers

Tool	Purpose
GCC	Compiles the C source into an executable (-Wall -Wextra for warnings, -std=c11).
GNU Make	Automates the build using a Makefile.
GDB	Debugging: breakpoints, stepping, inspecting variables and the process tree structures.
strace	Traces the system calls (openat, getdents64, read, clone/fork, wait4) made by the program.
ps / pstree	Reference tools used to verify that the project shows the same parent–child relations.
top / htop	Observes CPU and memory usage of the running tool in live mode.
Git / GitHub	Source control and sharing of the code among the team members.

CHAPTER 4: TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

C is the language of Unix and Linux, and almost all system call interfaces are defined for C. It gives direct control over memory and allows the program to use the kernel interface without any runtime layer. The project is written in C; the code is also valid for a C++ compiler apart from minor changes, and the abstract lists C/C++ as the implementation language.

Features of C used: structures (the Proc record), dynamic arrays with realloc(), pointers, recursion (tree printing), function modularisation, static and volatile variables, the standard library functions qsort() and bsearch(), and the preprocessor (#define _POSIX_C_SOURCE and #include).

4.2 GCC Compiler

GCC (GNU Compiler Collection) translates the C source into machine code. The project is compiled with warnings enabled to catch mistakes early:

```
gcc -Wall -Wextra -std=c11 -o ptree ptree.c
```

- -Wall -Wextra: enable common and extra warnings.
- -std=c11: use the C11 language standard.
- -o ptree: names the output executable. No -pthread flag is needed because the program uses no threads.

4.3 Linux System Calls

A system call is the controlled entry point through which a user program requests a service from the kernel. The following calls and library functions are used in the project:

System Call / Function	Header	Purpose
opendir(), readdir(), closedir()	<dirent.h>	List the entries of /proc (one folder per process).
open(), read(), close()	<fcntl.h>, <unistd.h>	Read /proc/PID/stat through a file descriptor.
sysconf(_SC_PAGESIZE)	<unistd.h>	Get the page size to convert RSS pages to KB.
fork(), _exit(), waitpid()	<unistd.h>, <sys/wait.h>	Create, end and reap child processes in the demo mode.
sigaction()	<signal.h>	Install the SIGINT handler reliably.
nanosleep()	<time.h>	Sleep between refreshes in live mode without busy-waiting.
getpid()	<unistd.h>	Find the root of the demo tree.
qsort(), bsearch()	<stdlib.h>	Library functions: sort the processes by PID and find a parent quickly.

4.4 POSIX Threads / Synchronization

POSIX Threads (pthreads) provide portable multithreading on Linux. This project is deliberately single-threaded: one thread scans /proc, builds the tree and prints it, so there is no shared data between threads and no mutex is required. Threads still appear in the project in one way: the number of threads of every process is read from the stat file (field num_threads) and shown with the -d option, which lets the learner see that a process such as a browser or a file-sync service may contain many threads.

Synchronisation in this project is concerned with the kernel rather than with threads. The process table changes at every moment, so a process seen by readdir() may finish before its stat file is opened. The scanner handles this race by checking the return value of open() and read() and silently skipping such a process. In the demo mode the parent uses waitpid() to synchronise with its children, which removes the zombie.

4.5 Linux Commands and Utilities

Command	Use
gcc / make	Compile and build the project.
./ptree	Run the program (options: -r PID, -d, -w SEC, -a, -D).
ps -ef / ps axjf	List processes with PPID; used to verify the tree.
pstree -p	Print the process tree with PIDs; used to compare with the project output.
cat /proc/PID/stat	Show the raw data that the program reads.
strace -f ./ptree	Watch the system calls made during execution.
top -p <pid>	Show CPU and memory usage of the running tool.
git add / commit / push	Maintain the GitHub repository.

4.6 Other Tools / Libraries Used

- VS Code and gedit: writing and editing source code.
- GitHub (github.com/udayasri-21/OSSP_PROJECT): hosting the repository and sharing code among the team.
- Standard C library (glibc): stdio, stdlib, string and ctype functions (printf, snprintf, sscanf, memcpy, strchr, strrchr, isdigit, atoi, getopt).
- Man pages (man 5 proc, man 2 fork, man 2 waitpid, man 3 opendir): reference documentation.

CHAPTER 5: SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The system follows a layered, modular architecture. The user starts the program from the terminal with optional command-line options. The main program parses the options, installs the SIGINT handler and calls the modules. The /proc scanner collects the data of all processes from the kernel, the tree builder links them into a tree, and the tree renderer prints the result. The demo module and the live (watch) mode are extra features built on the same modules. All services are finally provided by the Linux kernel through the /proc filesystem and system calls.

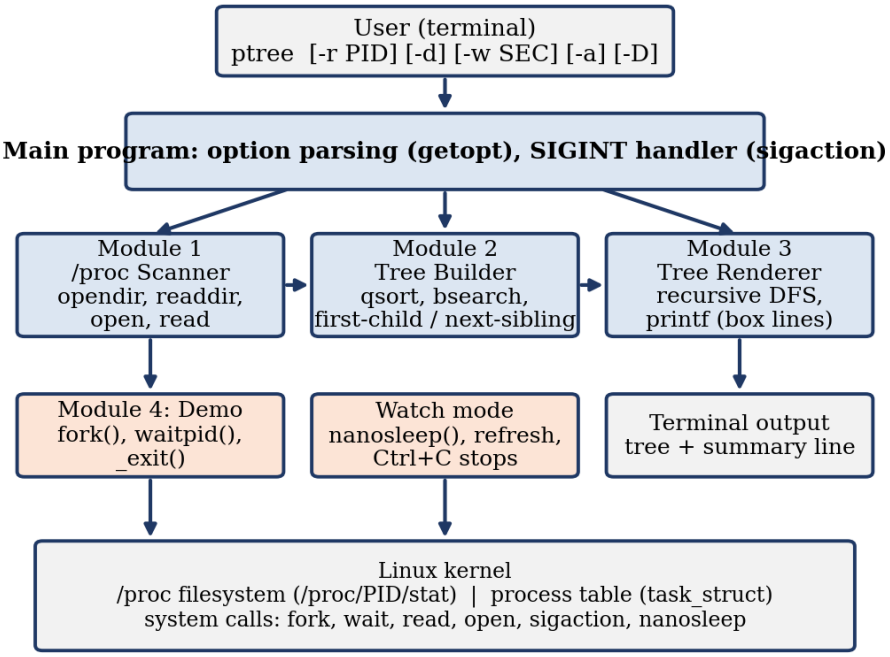


Figure 5.1: Overall system architecture of the Process Tree Visualizer

5.2 System Workflow

- Start: the program reads the command-line options with getopt() and installs the SIGINT handler with sigaction().
- If demo mode is selected, it creates a child, a grandchild and a zombie child with fork(), prints the tree below itself and then reaps the children with waitpid().
- Otherwise the scanner opens /proc and reads /proc/PID/stat of every numeric folder into an array.
- The array is sorted by PID and each process is linked to its parent using the PPID.
- The renderer prints every root and its descendants with a recursive traversal, marks zombies and prints a summary line.
- In live mode the screen is cleared, the tree is printed again after each nanosleep() interval, and the loop ends when SIGINT sets the flag running to 0.
- On exit the allocated memory is freed and the program ends.

5.3 Flowchart / Data Flow

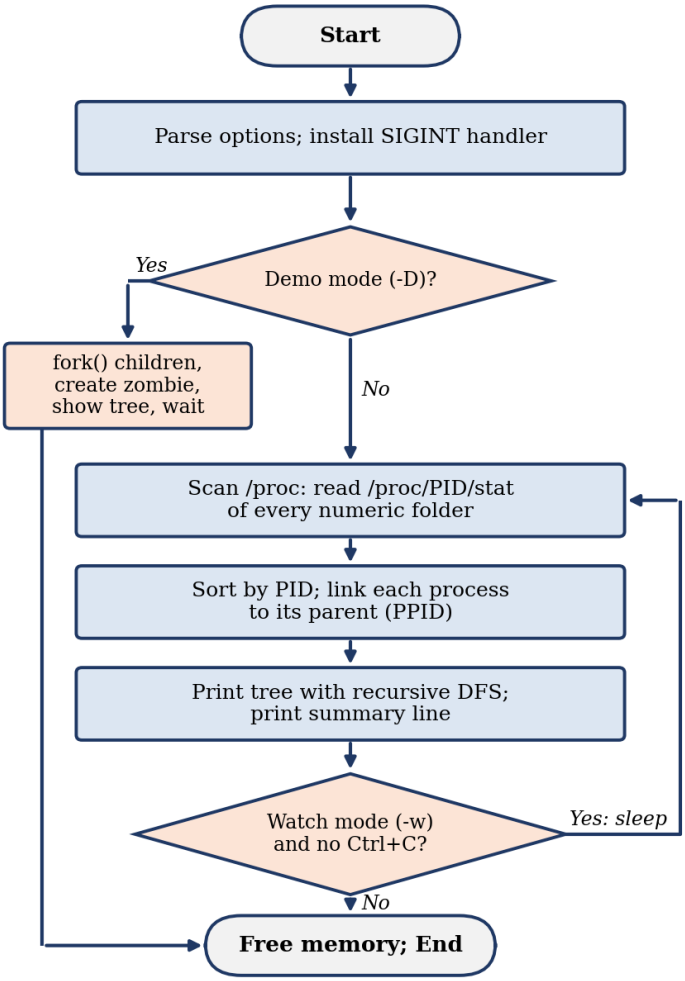


Figure 5.2: Flowchart of the Process Tree Visualizer

Data flow: /proc/PID/stat text (kernel) → Proc structure in memory → sorted array → parent–child links (first_child, next_sibling) → recursive traversal → tree lines and summary on the terminal.

5.4 Module Description

Module	Functions	Responsibility
Module 1: /proc Scanner	read_stat(), scan_proc()	Lists /proc, reads the stat file of each process, extracts PID, PPID, name, state, threads and RSS; skips processes that have already ended.
Module 2: Tree Builder	by_pid(), find_index(), build_tree()	Sorts the array by PID and links every process to its parent with first-child / next-sibling indices.
Module 3: Tree Renderer	print_node(), show_tree()	Recursive depth-first printing with branch lines, zombie marking, subtree selection and the summary line.
Module 4: Demo and Control	demo(), on_sigint(), main()	Option parsing, SIGINT handler, live (watch) mode, and the fork()/waitpid() demo tree.

5.5 Input and Output Design

Input design

Input	Format	Validation
-r PID	Positive integer	Zero, negative or non-numeric values give “invalid PID”; a PID that does not exist gives “no such process”.
-w SECONDS	Positive integer	Zero, negative or non-numeric values give “invalid interval”.
-d, -a, -D	Flags without value	Any unknown option prints the usage message.
/proc/PID/stat	Text line from the kernel	The name is read between the first “(” and the last “)”; if the line cannot be read or parsed, the process is skipped.

Output design

- One line per process in the form name(PID), indented with branch lines (├──, └──, │) or ASCII lines (|--, `--) with -a.
- With -d each line also shows [state, threads, memory in KB], for example [S, 1 thr, 1576 KB].
- Zombie processes are marked with the text “<-- ZOMBIE”.
- A summary line: “Processes scanned: N | Zombies shown: Z | Tree depth: D”.
- Error messages go to the standard error stream and the program returns exit code 1.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The program is implemented in a single source file, ptree.c, organised into the four modules described in Chapter 5. Each member of the team was responsible for one part of the implementation:

Roll Number	Student Name	Responsibility
2520090003	S Udayasri	Module 1 and 2: /proc scanning and parsing of the stat file; sorting and tree building.
2520090014	B Akshitha	Module 3: tree printing, command-line options and output format; testing.
2520090171	R Bhanu Priya	Module 4: demo mode (fork/waitpid), live mode and SIGINT handling; documentation.

Data structure used for each process (96 bytes per entry on a 64-bit system):

```
typedef struct {
    int  pid, ppid, threads;
    char state;
    char name[64];
    long rss_kb;
    int  first_child, next_sibling;   /* indices, -1 = none */
} Proc;
```

Instead of pointers, the tree uses array indices. Every process stores the index of its first child and the index of its next sibling, so any number of children needs only two links per process.

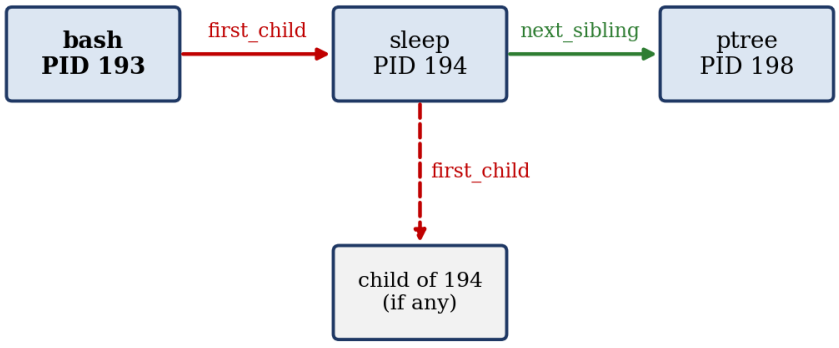


Figure 6.1: First-child / next-sibling representation of the process tree

6.2 Algorithm / Pseudocode

Main program:

```
BEGIN
  parse options with getopt()
  install SIGINT handler using sigaction()
  IF demo THEN demo()
  ELSE IF watch THEN
       WHILE running
          clear screen; show_tree(root)
          nanosleep(interval)
       END WHILE
  ELSE show_tree(root)
  free memory
END
```

show_tree(root):

```
scan_proc()                     /* read all processes from /proc */
build_tree()                    /* sort by PID, link parent-child */
IF root given THEN
   i <- find_index(root); IF not found THEN error
   print_node(i, depth 0)
ELSE
   FOR each process p
      IF parent of p is not in the list THEN print_node(p, depth 0)
print summary line
```

print_node(i, prefix, depth):

```
print prefix + branch symbol + name(PID)
IF details THEN print [state, threads, memory]
IF state = Z THEN print "<-- ZOMBIE"
newPrefix <- prefix + ("    " if last child else "|   ")
FOR each child c of i (via first_child, next_sibling)
   print_node(c, newPrefix, depth + 1)
```

6.3 Core C / C++ Implementation

Header files, constants and global data:

```
/* ptree.c - Linux Process Tree Visualizer
 * Reads /proc, builds the parent-child tree of running processes
 * and prints it in the terminal.  OSSP (25CS2104E) mini project.   */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <dirent.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <time.h>
#include <sys/wait.h>

typedef struct {
    int  pid, ppid, threads;
    char state;
    char name[64];
    long rss_kb;
    int  first_child, next_sibling;          /* indices, -1 = none */
} Proc;

static Proc *procs = NULL;
```

```
static int   nprocs = 0, cap = 0;
static int   show_detail = 0, use_ascii = 0, max_depth = 0, zombies = 0;
static volatile sig_atomic_t running = 1;

static void on_sigint(int sig) { (void)sig; running = 0; }
```

Module 1: /proc scanner (read_stat reads one process, scan_proc reads all):

```
/* ---- Module 1: /proc scanner ---------------------------------- */
static int read_stat(int pid, Proc *p) {
    char path[64], buf[1024];
    snprintf(path, sizeof path, "/proc/%d/stat", pid);
    int fd = open(path, O_RDONLY);
    if (fd < 0) return -1;                    /* process ended meanwhile */
    ssize_t n = read(fd, buf, sizeof buf - 1);
    close(fd);
    if (n <= 0) return -1;
    buf[n] = '\0';
    char *lp = strchr(buf, '('), *rp = strrchr(buf, ')');
    if (!lp || !rp || rp < lp) return -1;
    size_t len = (size_t)(rp - lp - 1);
    if (len >= sizeof p->name) len = sizeof p->name - 1;
    memcpy(p->name, lp + 1, len);
    p->name[len] = '\0';
    long rss_pages;
    if (sscanf(rp + 2, "%c %d %*d %*d %*d %*d %*u %*u %*u %*u %*u %*u %*u "
                       "%*d %*d %*d %*d %d %*d %*u %*u %ld",
               &p->state, &p->ppid, &p->threads, &rss_pages) != 4)
        return -1;
    p->pid = pid;
    p->rss_kb = rss_pages * (sysconf(_SC_PAGESIZE) / 1024);
    p->first_child = p->next_sibling = -1;
    return 0;
}

static void scan_proc(void) {
    DIR *d = opendir("/proc");
    if (!d) { perror("opendir /proc"); exit(1); }
    struct dirent *e;
    nprocs = 0;
    while ((e = readdir(d)) != NULL) {
        if (!isdigit((unsigned char)e->d_name[0])) continue;
        if (nprocs == cap) {
            cap = cap ? cap * 2 : 128;
            procs = realloc(procs, (size_t)cap * sizeof(Proc));
            if (!procs) { perror("realloc"); exit(1); }
        }
        if (read_stat(atoi(e->d_name), &procs[nprocs]) == 0) nprocs++;
    }
    closedir(d);
}
```

Module 2: tree builder:

```
/* ---- Module 2: tree builder ----------------------------------- */
static int by_pid(const void *a, const void *b) {
    return ((const Proc *)a)->pid - ((const Proc *)b)->pid;
}

static int find_index(int pid) {
    Proc key; key.pid = pid;
    Proc *r = bsearch(&key, procs, (size_t)nprocs, sizeof(Proc), by_pid);
    return r ? (int)(r - procs) : -1;
}

static void build_tree(void) {
```

```
    qsort(procs, (size_t)nprocs, sizeof(Proc), by_pid);
    for (int i = 0; i < nprocs; i++)
        procs[i].first_child = procs[i].next_sibling = -1;
    /* go backwards so that every child list ends up in PID order */
    for (int i = nprocs - 1; i >= 0; i--) {
        int par = find_index(procs[i].ppid);
        if (par >= 0 && par != i) {
            procs[i].next_sibling = procs[par].first_child;
            procs[par].first_child = i;
        }
    }
}
```

Module 3: tree renderer (recursive):

```
/* ---- Module 3: renderer ------------------------------------------ */
static void print_node(int i, const char *prefix, int is_last, int depth) {
    Proc *p = &procs[i];
    const char *mid  = use_ascii ? "|-- " : "\xe2\x94\x9c\xe2\x94\x80\xe2\x94\x80 ";
    const char *last = use_ascii ? "`-- " : "\xe2\x94\x94\xe2\x94\x80\xe2\x94\x80 ";
    const char *bar  = use_ascii ? "|   " : "\xe2\x94\x82   ";
    if (depth > max_depth) max_depth = depth;
    printf("%s%s%s(%d)", prefix, depth ? (is_last ? last : mid) : "",
           p->name, p->pid);
    if (show_detail)
        printf(" [%c, %d thr, %ld KB]", p->state, p->threads, p->rss_kb);
    if (p->state == 'Z') { printf("  <-- ZOMBIE"); zombies++; }
    putchar('\n');
    char next[512];
    snprintf(next, sizeof next, "%s%s", prefix,
             depth ? (is_last ? "    " : bar) : "");
    for (int c = p->first_child; c != -1; c = procs[c].next_sibling)
        print_node(c, next, procs[c].next_sibling == -1, depth + 1);
}

static int show_tree(int root_pid) {
    scan_proc();
    build_tree();
    max_depth = zombies = 0;
    if (root_pid > 0) {
        int r = find_index(root_pid);
        if (r < 0) {
            fprintf(stderr, "ptree: no such process: %d\n", root_pid);
            return -1;
        }
        print_node(r, "", 1, 0);
    } else {
        for (int i = 0; i < nprocs; i++)
            if (find_index(procs[i].ppid) < 0) print_node(i, "", 1, 0);
    }
    printf("\nProcesses scanned: %d | Zombies shown: %d | Tree depth: %d\n",
           nprocs, zombies, max_depth + 1);
    return 0;
}
```

Module 4: demo mode and main function:

```
/* ---- Module 4: demo (fork / wait) -------------------------------- */
static void demo(void) {
    pid_t c1 = fork();
    if (c1 == 0) {                          /* child 1 creates a grandchild */
        pid_t g = fork();
        if (g == 0) { sleep(3); _exit(0); }
        waitpid(g, NULL, 0);
        _exit(0);
    }
    pid_t c2 = fork();
```

```
    if (c2 == 0) _exit(0);        /* child 2 ends at once => zombie until waitpid */
    struct timespec ts = {0, 300000000L};
    nanosleep(&ts, NULL);
    show_tree((int)getpid());
    waitpid(c1, NULL, 0);                    /* reap children: zombie disappears */
    waitpid(c2, NULL, 0);
}

int main(int argc, char **argv) {
    int root = 0, watch = 0, opt, do_demo = 0;
    while ((opt = getopt(argc, argv, "r:dw:aDh")) != -1) {
        switch (opt) {
        case 'r':
            root = atoi(optarg);
            if (root <= 0) { fprintf(stderr, "ptree: invalid PID\n"); return 1; }
            break;
        case 'd': show_detail = 1; break;
        case 'w':
            watch = atoi(optarg);
            if (watch <= 0) {
                fprintf(stderr, "ptree: invalid interval\n");
                return 1;
            }
            break;
        case 'a': use_ascii = 1; break;
        case 'D': do_demo = 1; break;
        default:
            printf("Usage: %s [-r PID] [-d] [-w SECONDS] [-a] [-D]\n"
                   "  -r PID  show only the subtree of PID\n"
                   "  -d      show state, threads, memory\n"
                   "  -w SEC  refresh every SEC seconds (Ctrl+C to stop)\n"
                   "  -a      plain ASCII lines\n"
                   "  -D      demo: fork a small tree incl. a zombie\n", argv[0]);
            return opt == 'h' ? 0 : 1;
        }
    }
    struct sigaction sa = {0};
    sa.sa_handler = on_sigint;
    sigemptyset(&sa.sa_mask);
    sigaction(SIGINT, &sa, NULL);
    int rc = 0;
    if (do_demo) demo();
    else if (watch) {
        struct timespec ts = {watch, 0};
        while (running) {
            printf("\033[H\033[J");
            if (show_tree(root) < 0) { rc = 1; break; }
            fflush(stdout);
            nanosleep(&ts, NULL);            /* Ctrl+C interrupts the sleep */
        }
        printf("\nStopped.\n");
    } else rc = show_tree(root) < 0;
    free(procs);
    return rc;
}
```

6.4 Linux System Calls / APIs Used

The table below shows where each system call or API appears in the program.

API	Where used	Explanation
opendir(), readdir(), closedir()	scan_proc()	Walk through /proc; only folders whose names start with a digit are processes.
open(), read(), close()	read_stat()	Read /proc/PID/stat; a failure means the process has already ended and it is skipped.

API	Where used	Explanation
sscanf(), strchr(), strrchr()	read_stat()	Extract the name between the first “(” and the last “)”, then the state, PPID, threads and RSS fields.
sysconf(_SC_PAGESIZE)	read_stat()	Convert RSS from pages to kilobytes.
realloc()	scan_proc()	Doubles the process array when it is full (128, 256, ...).
qsort(), bsearch()	build_tree(), find_index()	Sort by PID and find the parent index in O(log n).
fork(), _exit(), waitpid()	demo()	Create child, grandchild and zombie; reap the children afterwards.
getpid()	demo()	PID of the demo root, used as the argument of show_tree().
sigaction()	main()	Registers on_sigint() as the SIGINT handler.
nanosleep()	main(), demo()	Waits between refreshes and lets the demo children start; interrupted by Ctrl+C.
getopt()	main()	Parses -r, -d, -w, -a and -D.

6.5 Process / Thread Implementation

When the program runs normally it is a single process with a single thread of execution. The process reads /proc, builds the tree and prints it, so its own PID is also visible in the output (for example ptree(198) as a child of the shell).

In demo mode the program creates real child processes:

- The demo process (parent) calls fork() to create child 1.
- Child 1 calls fork() again to create a grandchild, which sleeps for 3 seconds; child 1 waits for it with waitpid().
- The parent calls fork() a second time to create child 2, which exits immediately with _exit(0). Because the parent has not called waitpid() yet, child 2 stays in the zombie state (Z) with an entry in the process table.
- After a short pause of 0.3 s (nanosleep) the parent prints the tree rooted at its own PID, so the zombie is visible, and then calls waitpid() for both children, which removes the zombie.

The tree printed in this mode (parent → child → grandchild, plus the zombie child) is therefore the real result of the fork() calls, not a drawing, which makes it a good demonstration of the process hierarchy.

6.6 IPC / Synchronization Implementation

The program is single-threaded, so no mutex is needed. The following mechanisms are used for communication and synchronisation:

- Kernel to program (/proc): the kernel is the producer of the data and the program is the consumer. The data are read as text, one file per process, so no shared memory has to be locked by the program.

- Signal (SIGINT): when the user presses Ctrl+C in live mode the kernel delivers SIGINT. The handler only sets the flag running to 0 (declared volatile sig_atomic_t), which is a safe operation in a signal handler; the main loop then ends normally and frees memory.
- Parent–child synchronisation (waitpid): in demo mode the parent waits for its children, which collects their exit status and removes the zombie.
- Race with exiting processes: because processes can end at any time, a process listed by readdir() may be gone when open() is called. The scanner checks every return value and skips such processes.

The tree is a snapshot: processes created or ended during the short scan may be missing or may point to a parent that has just ended. In the second case the process is shown as a separate root, which is acceptable for a visualization tool.

6.7 Memory Management Implementation

- Dynamic allocation: the array of Proc structures grows with realloc() (128, 256, 512, ... entries). Each entry takes 96 bytes, so even 1000 processes need only about 96 KB. The array is released with free() at the end, and the return value of realloc() is checked.
- Bounded buffers: process names are limited to 63 characters (memcpy with a length check), the stat line is read into a 1024-byte buffer with the terminating zero added by the program, and file paths are built with snprintf() so that no overflow is possible.
- Index-based tree: parent and child links are array indices (not pointers), so they remain valid even after realloc() moves the array.
- Stack usage: the recursive print_node() uses a 512-byte prefix buffer per level; the depth of a process tree is normally below 20, so the stack use is small. snprintf() truncates the prefix safely if the tree were extremely deep.
- Resource release: the directory is closed with closedir() and every file descriptor with close() immediately after use.

6.8 Optional User Interface Implementation

The user interface is a command-line interface with options, which fits the goal of a lightweight tool:

```
Usage: ./ptree [-r PID] [-d] [-w SECONDS] [-a] [-D]
  -r PID  show only the subtree of PID
  -d      show state, threads, memory
  -w SEC  refresh every SEC seconds (Ctrl+C to stop)
  -a      plain ASCII lines
  -D      demo: fork a small tree incl. a zombie
```

In live mode the screen is cleared with the ANSI sequence ESC[H ESC[J before every refresh, so the tree stays at the same place on the screen. A graphical front end, colours, searching by name or an interactive tree (with ncurses) could be added later without changing the scanning and tree-building modules.

CHAPTER 7: EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

Open a terminal in the project folder (cloned from https://github.com/udayasri-21/OSSP_PROJECT) and compile:

```
gcc -Wall -Wextra -std=c11 -o ptree ptree.c
```

The program compiled without any errors or warnings on Ubuntu with GCC. The Makefile used by the team is:

```
CC = gcc
CFLAGS = -Wall -Wextra -std=c11

ptree: ptree.c
      $(CC) $(CFLAGS) -o ptree ptree.c

clean:
      rm -f ptree
```

7.2 Execution Commands

```
./ptree                 # full process tree of the system
./ptree -d              # with state, threads and memory
./ptree -r 1234          # only the subtree of PID 1234
./ptree -a              # plain ASCII lines
./ptree -w 2              # refresh every 2 seconds (Ctrl+C to stop)
./ptree -D -d           # demo: fork a tree with a zombie process
pstree -p               # compare with the standard tool
strace -f ./ptree       # observe the system calls
make clean              # remove the executable
```

7.3 Sample Inputs and Outputs

The following sessions were captured by running the program on Linux. The process IDs and names are different on every computer and on every run.

Session 1: demo mode with details (-D -d)

The demo process creates a child, a grandchild and a second child that exits at once. The second child is shown as a zombie because its parent has not yet called waitpid().

```
$ ./ptree -D -d
ptree(185) [R, 1 thr, 1688 KB]
├── ptree(186) [S, 1 thr, 260 KB]
│    └── ptree(188) [S, 1 thr, 128 KB]
└── ptree(187) [Z, 1 thr, 0 KB]  <-- ZOMBIE

Processes scanned: 55 | Zombies shown: 1 | Tree depth: 3
```

Session 2: subtree of a shell with details (-d -r PID)

A shell was started with two background sleep commands; the visualizer was run from the same shell, so it appears as the third child.

```
$ ./ptree -d -r 193
bash(193) [S, 1 thr, 3240 KB]
├── sleep(194) [S, 1 thr, 1576 KB]
├── sleep(195) [S, 1 thr, 1576 KB]
└── ptree(198) [R, 1 thr, 1684 KB]

Processes scanned: 56 | Zombies shown: 0 | Tree depth: 2
```

Session 3: ASCII mode (-a -r PID)

```
$ ./ptree -a -r 199
bash(199)
|-- sleep(200)
|-- sleep(201)
`-- ptree(202)

Processes scanned: 56 | Zombies shown: 0 | Tree depth: 2
```

Session 4: invalid input

```
$ ./ptree -r 99999
ptree: no such process: 99999
$ ./ptree -r abc
ptree: invalid PID
$ ./ptree -w 0
ptree: invalid interval
```

Test cases

Test Case	Input	Expected Result	Result
Full tree	./ptree	All processes printed as a tree with a summary line	Pass
Details	./ptree -d	State, threads and memory shown for each process	Pass
Subtree	./ptree -r <PID of shell>	Only that process and its descendants shown	Pass
Non-existing PID	-r 99999	“no such process” message, exit code 1	Pass
Invalid PID	-r abc	“invalid PID” message	Pass
Invalid interval	-w 0	“invalid interval” message	Pass
ASCII mode	-a	Lines drawn with |-- and `--	Pass
Zombie detection	-D -d	One child marked ZOMBIE; no zombie after the run	Pass
Live mode and Ctrl+C	-w 1, then Ctrl+C	Tree refreshed every second; “Stopped.” printed and program ends cleanly	Pass
Cross-check	./ptree and pstree -p	Same parent–child relations	Pass

7.4 Screenshots of Execution

Replace each box below with a screenshot taken from your own terminal while running the command shown.

[Insert screenshot here]

Command: `gcc -Wall -Wextra -std=c11 -o ptree ptree.c`

Figure 7.1: Compilation of ptree.c with GCC

[Insert screenshot here]

Command: `./ptree -d`

Figure 7.2: Full process tree with details

[Insert screenshot here]

Command: `./ptree -D -d`

Figure 7.3: Demo mode showing a zombie process

[Insert screenshot here]

Command: `./ptree -r <PID> and pstree -p <PID>`

Figure 7.4: Subtree view compared with pstree

7.5 Demonstration of OS Concepts

OS Concept	How it is demonstrated
Process hierarchy	The printed tree shows that every process has exactly one parent; the shell is the parent of every command started from it, and PID 1 is the ancestor of the user processes.
fork() and wait()	In demo mode the tree shows parent, child and grandchild created by fork(); after waitpid() the children disappear from the table.
Zombie process	The demo shows a child in state Z with 0 KB of memory: it has finished but its entry stays until the parent calls waitpid().

OS Concept	How it is demonstrated
Process states	With -d the letters R (running), S (sleeping), I (idle kernel thread) and Z (zombie) are visible next to each process.
/proc filesystem	ls /proc and cat /proc/PID/stat show the same data that the program reads; nothing is stored on disk.
Threads	Programs with several threads (for example file-sync or browser processes) show a thread count greater than 1 with -d.
Signals	In live mode, Ctrl+C sends SIGINT and the handler ends the loop cleanly with the message “Stopped.”
System calls	strace -f ./ptree shows openat, getdents64, read, close, clone/fork, wait4 and nanosleep issued by the program.

CHAPTER 8: CONCLUSION

8.1 Summary of the Project

The project successfully designed and implemented a Linux-based command-line Process Tree Visualizer in C using POSIX system calls. The tool reads the process information of the whole system from the /proc filesystem, builds the parent–child tree with a first-child / next-sibling structure, and prints it with a recursive traversal. Options for details, subtree selection, ASCII output and live refresh make it flexible, and the demo mode shows how fork() and waitpid() create and remove a process hierarchy including a zombie. The program uses very little memory and no external library.

8.2 Objectives Achieved

Objective	Status	How it was achieved
Design a Linux-based command-line process tree visualizer in C using POSIX system calls	Achieved	ptree.c with four modules and command-line options.
Collect process information from the /proc filesystem	Achieved	opendir(), readdir(), open() and read() on /proc/PID/stat for every process.
Build and display the parent–child hierarchy using a tree and recursive traversal	Achieved	Sorting with qsort(), linking with bsearch(), printing with print_node().
Demonstrate fork, wait, zombie state and signal handling	Achieved	Demo mode (-D) with fork()/waitpid() and live mode (-w) with a SIGINT handler.

8.3 Results Achieved

- A working tool that shows the complete process tree of a Linux system, or the subtree of any chosen process.
- Correct parent–child relations, verified against the output of ps and pstree.
- Zombie processes are detected and marked, and the demo mode shows when they appear and disappear.
- Useful extra information (state, threads, memory) and a summary of the number of processes and the depth of the tree.
- Small program: about 200 lines of C, a binary of about 22 KB and 96 bytes of memory per process, without any extra library.
- Source code and documentation are available in the GitHub repository.

Limitations: the tree is a snapshot, so processes that start or end during the scan may be missing; only the process name is shown (not the full command line or the user); CPU usage is not displayed; each thread is not listed separately; and the tool works only on Linux because it depends on /proc.

Future scope: show the full command line (/proc/PID/cmdline) and the owner, colour the output by state, search or filter by name, add an interactive view with ncurses, export the tree to Graphviz or JSON, show control groups and containers, and allow sending signals to a selected process.

8.4 Skills and Learning Outcomes

- Practical understanding of processes, PID/PPID, process states, zombies, the /proc filesystem, system calls and signals in Linux.
- Skill in writing, compiling, debugging (GDB) and tracing (strace) C programs on Ubuntu.
- Use of tree data structures, sorting, binary search and recursion in a systems-programming problem.
- Awareness of safe coding: input validation, bounded buffers, checking return values and handling races with exiting processes.
- Teamwork, task division and version control with Git and GitHub.
- Technical documentation skills.

CHAPTER 9: REFERENCES

9.1 Books

1. A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018.
2. M. Kerrisk, The Linux Programming Interface: A Linux and UNIX System Programming Handbook, No Starch Press, 2010.
3. W. R. Stevens and S. A. Rago, Advanced Programming in the UNIX Environment, 3rd ed., Addison-Wesley, 2013.
4. A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed., Pearson, 2014.
5. B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed., Prentice Hall, 1988.

9.2 Research Papers

6. D. M. Ritchie and K. Thompson, “The UNIX Time-Sharing System,” Communications of the ACM, vol. 17, no. 7, pp. 365–375, 1974.
7. IEEE and The Open Group, “IEEE Std 1003.1 (POSIX.1-2008): Base Specifications, Issue 7,” 2008.

9.3 Linux / GNU Documentation

8. Linux man-pages: proc(5), fork(2), wait(2), execve(2), exit(2). https://man7.org/linux/man-pages/
9. Linux man-pages: opendir(3), readdir(3), open(2), read(2), sysconf(3), getopt(3), qsort(3), bsearch(3). https://man7.org/linux/man-pages/
10. Linux man-pages: signal(7), sigaction(2), nanosleep(2), ps(1), pstree(1). https://man7.org/linux/man-pages/
11. The Linux Kernel documentation, “The /proc Filesystem.” https://docs.kernel.org/filesystems/proc.html
12. GNU C Library Reference Manual. https://www.gnu.org/software/libc/manual/
13. GCC, the GNU Compiler Collection – Online Documentation. https://gcc.gnu.org/onlinedocs/
14. GNU Make Manual. https://www.gnu.org/software/make/manual/

9.4 Online Resources

15. Project source code, GitHub repository: https://github.com/udayasri-21/OSSP_PROJECT
16. Ubuntu documentation. https://ubuntu.com/server/docs
17. Microsoft, “Windows Subsystem for Linux Documentation.” https://learn.microsoft.com/windows/wsl/